

LAB4_REPORT_Multi Cycle CPU

컴퓨터소프트웨어학부 2021026108 김민제

구현 환경

운영체제 : macOS Sonoma

verilog : iverilog

vcd : gtkwave

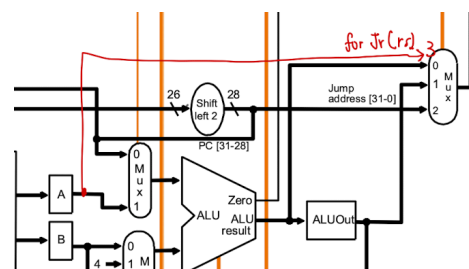
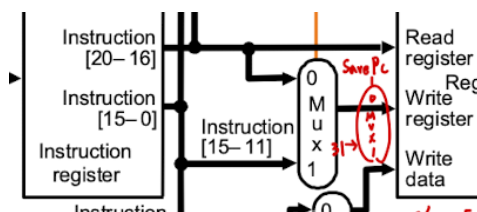
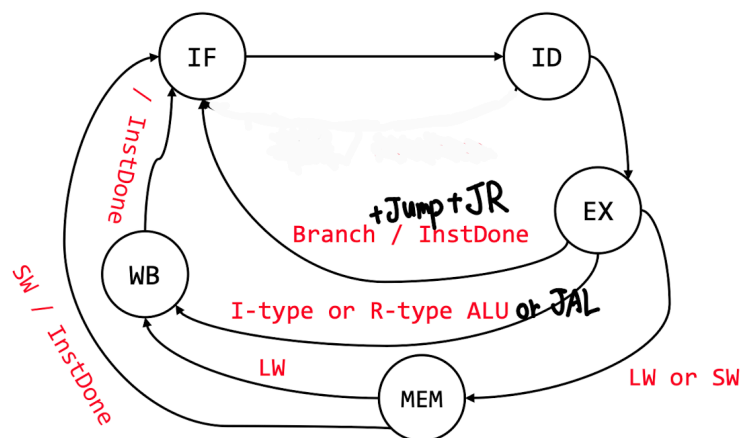
cpp : Apple clang version 16.0.0 (clang-1600.0.26.6)

지난 과제 파일

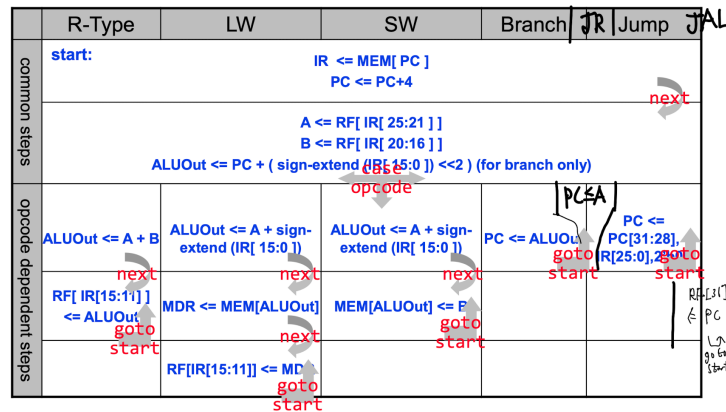
ALU.cpp ALU.h ALU.v RF.cpp RF.h RF.v 는 지난 과제 제출과 동일한 파일을 사용하였습니다.

전체 구조 분석 및 설계

지난 과제들로 만든 SingleCycle CPU에서 FSM을 만들어 여러 단계로 나누어 각 명령어마다 다른 STATE들을 돌도록하여 명령어별 cycle을 다르게 해주어 CPI를 줄이는 MultiCycle을 구현하는 것이 목표입니다. 이를 구현하기 위해서 Controller에 각 단계를 기억하도록 하고 각 단계별로 control값을 다르게 뿌리도록 하여 각 단계마다 적절한 하드웨어가 동작하여 값을 출력하도록 하였습니다. 먼저 전체적인 동작을 생각해보면 IF→ID→EX→MEM→WB 단계를 거친다고 하였을 때 1tick 마다 한 단계씩 동작하게 됩니다. 이때 모든 resource를 공유하도록 하여 각 단계에서 단 한 번만 모듈들이 동작하도록 했습니다. 이론수업에서 배운 내용은 각 단계별로 실행되는 단계, 하드웨어의 부분이 다른 것처럼 공부하였지만 실제의 HW의 동작의 경우 clk이 될 때 모든 모듈들이 동작하는데 이를 컨트롤 값을 조절하여 논리적으로 한 틱에 하나의 단계만 실행하도록 구현했습니다. 이를 위해 과제 명세에 있는 FSM과 Microarchitecture 설계도에 JR과 JAL을 추가하여 그렸습니다.



Combined RT Sequencing



CPP 설계 및 구현

I. globals.h

FSM state를 관리하기 쉽게 하기 위해 enum으로 값들을 정의했습니다.

```
enum FSMState {
    STATE_IF = 1,
    STATE_ID = 2,
    STATE_EX = 3,
    STATE_MEM = 4,
    STATE_WB = 5,
    STATE_ERROR = 6, // FOR ERROR ⇒ 사용은 안함
};
```

II. CTRL.h

MicroArchitecture 설계도에 있는 새로운 컨트롤 값들을 추가해주고 state 관리를 위한 변수와 함수들을 추가해주었습니다.

```
void resetFSM();
void nextState(uint32_t opcode, uint32_t funct);

uint32_t state;
ParsedInst inst;
```

III. CTRL.cpp

1. resetFSM()은 초기 state를 IF 단계로 초기화해주는 함수입니다.
2. nextState()은 각 명령어에 따라서 FSM의 단계의 이동을 결정해주는 함수입니다.
3. controlSignal()은 명령어와 각 state를 보고 알맞은 signal 값을 내보내는 함수입니다.
 - a. STATE_IF 에서는 $IR \leq MEM[PC]$, $PC \leq PC+4$ 를 하기 위해서 아래와 같이 값을 세팅합니다.

```
case STATE_IF:
    controls→MemRead = 1;
    controls→MemWrite = 0;
    controls→IRWrite = 1;
    controls→PCWrite = 1;
    controls→ALUSrcA = 0;
    controls→ALUSrcB = 1;
```

```

controls→PCSource = 0;
controls→ALUOp = ALU_ADDU;
break;

```

- b. STATE_ID에서는 $A \leftarrow RF[IR[25:21]]$, $B \leftarrow RF[IR[20:16]]$, $ALUOut \leftarrow PC + \text{sign}(IR[15:0]) \ll 2$ 를 하기 위해서 아래와 같이 컨트롤을 세팅했습니다.

```

case STATE_ID:
controls→ALUSrcA = 0;
controls→ALUSrcB = 3;
controls→SignExtend = 1;
controls→ALUOp = ALU_ADDU;
break;

```

- c. STATE_EX에서는 ALUOut에 해당하는 연산을 해주거나 target으로 PC를 업데이트 해주기 위해서 아래와 같이 값을 세팅했습니다.

```

switch(opcode) {
case OP_RTYPE: // R-type Instruction
controls→RegDst = 1; // use rd
controls→ALUSrcA = 1; // use rs
controls→ALUSrcB = 0; // use rt
switch (funct) {
case FUNCT_ADDU:
controls→ALUOp = ALU_ADDU;
break;
...
}
case OP_J: // J target
controls→PCSource = 2;
controls→PCWrite = 1;
break;
case OP_JAL: // JAL target
controls→PCSource = 2;
controls→PCWrite = 1;
break;
case OP_BEQ: // BEQ rs, rt, offset
controls→ALUSrcA = 1;
controls→ALUSrcB = 0;
controls→PCWriteCond = 1;
controls→PCSource = 1;
controls→ALUOp = ALU_EQ;
break;
case OP_BNE: // BNE rs, rt, offset
controls→ALUSrcA = 1;
controls→ALUSrcB = 0;
controls→PCWriteCond = 1;
controls→PCSource = 1;
controls→ALUOp = ALU_NEQ;
break;
case OP_ADDIU: // ADDIU rt, rs, imm
controls→ALUSrcA = 1;
controls→ALUSrcB = 2;
controls→SignExtend = 1;
controls→ALUOp = ALU_ADDU;

```

```

        break;
    case OP_ANDI: // ANDI rt, rs, imm
        controls→ALUSrcA = 1;
        controls→ALUSrcB = 2;
        controls→SignExtend = 0;
        controls→ALUOp = ALU_AND;
        break;
    case OP_LUI: // LUI rt, imm
        controls→ALUSrcA = 1;
        controls→ALUSrcB = 2;
        controls→SignExtend = 0;
        controls→ALUOp = ALU_LUI;
        break;
    case OP_ORI: // ORI rt, rs, imm
        controls→ALUSrcA = 1;
        controls→ALUSrcB = 2;
        controls→SignExtend = 0;
        controls→ALUOp = ALU_OR;
        break;
    case OP_SLTI: // SLTI rt, rs, imm
        controls→ALUSrcA = 1;
        controls→ALUSrcB = 2;
        controls→SignExtend = 1;
        controls→ALUOp = ALU_SLT;
        break;
    case OP_SLTIU: // SLTIU rt, rs, imm
        controls→ALUSrcA = 1;
        controls→ALUSrcB = 2;
        controls→SignExtend = 1;
        controls→ALUOp = ALU_SLTU;
        break;
    case OP_XORI: // XORI rt, rs, imm
        controls→ALUSrcA = 1;
        controls→ALUSrcB = 2;
        controls→SignExtend = 0;
        controls→ALUOp = ALU_XOR;
        break;
    case OP_LW: // LW, rt, offset(rs)
        controls→ALUSrcA = 1;
        controls→ALUSrcB = 2;
        controls→SignExtend = 1;
        controls→ALUOp = ALU_ADDU;
        break;
    case OP_SW: // Sw rt, offset(rs)
        controls→ALUSrcA = 1;
        controls→ALUSrcB = 2;
        controls→SignExtend = 1;
        controls→ALUOp = ALU_ADDU;
        break;
    default:
        status = INVALID_INST; // not valid instruction - error detect
        break;
}

```

- d. STATE_MEM에서는 LW는 MEM을 읽고 Data를 가져오는 것이므로 lorD를 1로 설정했고 SW는 MEM에 쓰는 것이고 Data에 접근하는 것이므로 lorD를 1로 설정했습니다.

```
case STATE_MEM:
    if(opcode==OP_LW){
        controls→MemRead = 1;
        controls→lorD = 1;
    }else if(opcode==OP_SW){
        controls→MemWrite = 1;
        controls→lorD = 1;
    }
    break;
```

- e. STATE_WB에서는 레지스터에 값을 쓰는 동작을 하므로 이를 위해서 값을 아래처럼 세팅했습니다.

```
case STATE_WB:
    if(opcode==OP_RTYPE){
        controls→RegWrite = 1;
        controls→MemtoReg = 0;
        controls→RegDst = 1;
    }else if(opcode==OP_SW){
        controls→RegWrite = 0;
        controls→MemtoReg = 0;
    }else if(opcode==OP_LW){
        controls→RegWrite = 1;
        controls→MemtoReg = 1;
    }else if(opcode==OP_JAL){
        controls→SavePC = 1;
        controls→RegWrite = 1;
    }else if(opcode==OP_LUI || opcode==OP_ORI || opcode==OP_ANDI
    || opcode==OP_SLTI || opcode==OP_XORI || opcode==OP_ADDIU
    || opcode==OP_SLTIU){
        controls→RegWrite = 1;
        controls→MemtoReg = 0;
    }
    break;
```

4. splitinst(), signExtend는 지난번 과제와 동일합니다.

IV. CPU.h

1. multicycle 구현을 위해서 필요한 저장소(IR, A, B, ALUOut, MDR)들을 추가로 구현했습니다.

V. CPU.cpp

1. init() 함수에서 새로 추가한 저장소들의 값을 0으로 초기화했습니다. 그리고 ctrl의 resetFSM()을 호출하여 state을 초기화했습니다.
2. tick() 함수 구현
 - a. branch를 위한 zero 를 추가했습니다.
 - b. 제일 먼저 IR을 split하고 이를 가지고 컨트롤 값을 생성했습니다.
 - c. 이후 lorD를 통해 메모리에 접근하여 Inst를 읽을지 memory data를 읽을지 결정합니다.
 - d. 값이 준비가 되었으므로 memAccess로 메모리에서 값을 가져옵니다.
 - e. 이때 mem에서 읽어온 값이 0이라면 status를 종료로 바꾸고 종료합니다. (원래는 Mem에 종료분기가 있었지만 i와 d를 합치면서 사라져서 임의로 추가했습니다.)

- f. rf.read로 레지스터값을 읽어옵니다.
- g. operand1, operand2, immi 값을 생성해주고 alu.compute로 alu 연산을 합니다.
- h. zero를 세팅하고 PCWrite 이거나 PCWriteCond 이고 zero라면 PC 값을 업데이트합니다.
- i. JAL 명령어라면 wr_addr = 31, wr_data = ALUOut, 아니면 RegDst, MemtoReg 컨트롤 값에 따라서 다르게 처리합니다.
- j. rf.write으로 레지스터에 값을 씁니다.
- k. 마지막으로 microArch 값들을 세팅해줍니다.
- l. 그후 다음 State로 넘어가도록 하는 nextState()를 실행시킵니다.

CPP 동작 확인

I. cpp console log

1. memory 확인을 위해 MEM.cpp에 MEM::dump() 함수를 만들어 gp부터 쓰인 값이 있다면 읽어오도록 했습니다.

```
void MEM::dump() {
    for(uint32_t a = (0x1800>>2); a < MEMSIZE; ++a){
        if(memory[a] != 0){
            cout << hex << setw(8) << setfill('0') << (a << 2) << " : " << memory[a] << "\n";
        }
    }
}
```

2. CPU.cpp 에서 break 포인트를 걸어주는 명령어와 r을 입력했을때 레지스터 뿐만 아니라 메모리에 값이 있다면 그 값도 읽어오도록 했습니다.

```
case 'b':
    while(cpu.PC!=0x630) cpu.tick();
    break;
case 'r':
    cpu.rf.dump();
    cpu.mem.dump();
    break;
```

3. 종료될 때 역시 레지스터와 메모리 모두 출력하도록 했습니다.

```
cout << "RF states after the program execution" << endl;
cpu.rf.dump();
cpu.mem.dump();
```

4. 아래는 각 테스트 케이스의 MARs와 cpp console log를 비교한 결과입니다.
 - a. testcase 1번과 6번의 레지스터와 메모리를 비교한 것입니다.

II. CTRL.v

1. cpp과 동일하게 multicycle 구현을 위해 필요한 새로운 컨트롤 값들을 추가해주었습니다.
2. reg [4:0] state, nextState; 로 state를 관리하도록 하였습니다.
3. clk이 될때마다 state<=nextState로 설정해주고 rst 라면 각각 IF와 ID로 세팅하도록 했습니다.
4. always @(*) 에서 먼저 명령어에 따라서 state를 변경해줍니다.

```
case (state)
  `STATE_IF: begin
    nextState <= `STATE_ID;
  end
  `STATE_ID: begin
    nextState <= `STATE_EX;
  end
  `STATE_EX: begin
    case (opcode)
      `OP_BEQ, `OP_BNE, `OP_J: begin
        nextState <= `STATE_IF; // BEQ, BNE, J
      end
      `OP_RTYPE: begin
        if(funcnt == `FUNCT_JR) nextState <= `STATE_IF;
        else nextState <= `STATE_WB;
      end
      `OP_ADDIU, `OP_LUI, `OP_ORI, `OP_ANDI, `OP_SLTI, `OP_XORI, `OP_SLTIU, `OP_JAL: nextState <= `STATE_WB;
      default: nextState <= `STATE_MEM;
    endcase
  end
  `STATE_MEM: begin
    if(opcode == `OP_SW)
      nextState <= `STATE_IF;
    else if (opcode == `OP_LW) // LW
      nextState <= `STATE_WB;
    else
      nextState <= `STATE_WB;
    end
  end
  `STATE_WB: nextState <= `STATE_IF;
endcase
```

5. 이후 컨트롤 값을 0으로 초기화해줍니다.
6. 단계에 따라서 컨트롤 값을 세팅합니다.
 - a. IF 단계라면

```
`STATE_IF: begin
  MemRead = 1;
  IRWrite = 1;
  PCWrite = 1;
  ALUSrcA = 0;
  ALUSrcB = 1;
  ALUOp = `ALU_ADDU;
end
```

- b. ID 단계라면


```

`STATE_ID: begin
    ALUSrcA = 0;
    ALUSrcB = 3;
    SignExtend = 1;
    ALUOp = `ALU_ADDU;
end

```

c. EX 단계라면

```

`STATE_EX: begin
    case (opcode)
        `OP_RTYPE: begin
            RegDst = 1;
            ALUSrcA = 1;
            ALUSrcB = 0;
            case (funct)
                `FUNCT_ADDU: ALUOp = `ALU_ADDU;
                `FUNCT_AND: ALUOp = `ALU_AND;
                `FUNCT_NOR: ALUOp = `ALU_NOR;
                `FUNCT_OR: ALUOp = `ALU_OR;
                `FUNCT_SLT: ALUOp = `ALU_SLT;
                `FUNCT_SLTU: ALUOp = `ALU_SLTU;
                `FUNCT_SUBU: ALUOp = `ALU_SUBU;
                `FUNCT_XOR: ALUOp = `ALU_XOR;
                `FUNCT_SLL: ALUOp = `ALU_SLL;
                `FUNCT_SRA: ALUOp = `ALU_SRA;
                `FUNCT_SRL: ALUOp = `ALU_SRL;
                `FUNCT_JR: begin
                    PCSource = 3;
                    PCWrite = 1;
                end
            endcase
        end
        `OP_J: begin
            PCSource = 2;
            PCWrite = 1;
        end
        `OP_JAL: begin
            PCSource = 2;
            PCWrite = 1;
        end
        `OP_BEQ: begin
            ALUSrcA = 1;
            ALUSrcB = 0;
            PCWriteCond = 1;
            PCSource = 1;
            ALUOp = `ALU_EQ;
        end
        `OP_BNE: begin
            ALUSrcA = 1;
            ALUSrcB = 0;
            PCWriteCond = 1;
            PCSource = 1;
            ALUOp = `ALU_NEQ;
        end
    endcase
end

```

```

end
`OP_ADDIU: begin
    ALUSrcA = 1;
    ALUSrcB = 2;
    SignExtend = 1;
    ALUOp = `ALU_ADDU;
end
`OP_ANDI: begin
    ALUSrcA = 1;
    ALUSrcB = 2;
    SignExtend = 0;
    ALUOp = `ALU_AND;
end
`OP_LUI: begin
    ALUSrcA = 1;
    ALUSrcB = 2;
    SignExtend = 0;
    ALUOp = `ALU_LUI;
end
`OP_ORI: begin
    ALUSrcA = 1;
    ALUSrcB = 2;
    SignExtend = 0;
    ALUOp = `ALU_OR;
end
`OP_SLT: begin
    ALUSrcA = 1;
    ALUSrcB = 2;
    SignExtend = 1;
    ALUOp = `ALU_SLT;
end
`OP_SLTIU: begin
    ALUSrcA = 1;
    ALUSrcB = 2;
    SignExtend = 1;
    ALUOp = `ALU_SLTU;
end
`OP_XORI: begin
    ALUSrcA = 1;
    ALUSrcB = 2;
    SignExtend = 0;
    ALUOp = `ALU_XOR;
end
`OP_LW: begin
    ALUSrcA = 1;
    ALUSrcB = 2;
    SignExtend = 1;
    ALUOp = `ALU_ADDU;
end
`OP_SW: begin
    ALUSrcA = 1;
    ALUSrcB = 2;
    SignExtend = 1;
    ALUOp = `ALU_ADDU;
end
end

```

```

        default: begin
            // ALUOp = `ALU_NOP;
            // invalid instruction, do nothing (or signal error)
        end
    endcase
end

```

d. MEM 단계라면

```

`STATE_MEM: begin
    case (opcode)
        `OP_LW: begin
            MemRead = 1;
            lorD = 1;
        end
        `OP_SW: begin
            MemWrite = 1;
            lorD = 1;
        end
    endcase
end

```

e. WB 단계라면

```

`STATE_WB: begin
    case (opcode)
        `OP_RTYPE: begin
            RegWrite = 1;
            MemtoReg = 0;
            RegDst = 1;
        end
        `OP_SW: begin
            RegWrite = 0;
            MemtoReg = 0;
        end
        `OP_LW: begin
            RegWrite = 1;
            MemtoReg = 1;
        end
        `OP_JAL: begin
            SavePC = 1;
            RegWrite = 1;
        end
        `OP_ADDIU, `OP_LUI, `OP_ORI, `OP_ANDI, `OP_SLTI,
        `OP_XORI, `OP_SLTIU: begin
            RegWrite = 1;
            MemtoReg = 0;
        end
    endcase
end

```

III. CPU.v 구현

1. MicroArch를 추가해줍니다.

```

reg [31:0]    PC;
reg [31:0]    IR;
reg [31:0]    A;
reg [31:0]    B;
reg [31:0]    ALUOut;
reg [31:0]    MDR;

```

2. 매 클락마다 MicroArch를 업데이트 해줍니다. Memory가 하나로 합쳐져서 inst를 읽을지 memory data를 읽을지 컨트롤 값에 따라서 달라지므로 IRWrite에 따라서 다르게 저장합니다. PC의 경우 설계와 동일하게 컨트롤값들과 alu_zero의 값들의 조합으로 PC를 업데이트할지 말지 결정하고 PCSource로 알맞은 값을 연결해줍니다.

```

always @(posedge clk) begin
    if(rst) begin
        PC <= 0;
        IR <= 0;
        A <= 0;
        B <= 0;
        ALUOut <= 0;
        MDR <= 0;
    end else begin
        if(IRWrite) begin
            IR <= mem_read_data;
        end else
            MDR <= mem_read_data;
        A <= rd_data1;
        B <= rd_data2;
        ALUOut <= alu_result;

        if (PCWrite || (PCWriteCond && alu_zero)) begin
            case(PCSource)
                0: PC <= alu_result;
                1: PC <= ALUOut;
                2: PC <= {PC[31:28], immj, 2'b00};
                3: PC <= A;
            endcase
        end
    end
end

```

3. assign 문들로 IR을 split 해주고, signExtend 값을 만들어주고, lrd를 가지고 mem에서 읽을 주소를 정합니다.

```

assign ext_imm = SignExtend ? {{16{immi[15]}}, immi} : {16'b0, immi};
assign mem_addr = lrd?ALUOut:PC;

```

4. CTRL, MEM, RF 모듈을 연결해줍니다.
5. assign 문으로 operand1, operand2를 설정해줍니다.

```

assign operand1 = ALUSrcA?A:PC;
assign operand2 = (ALUSrcB==0)?B:
    (ALUSrcB==1)?32'd4:
    (ALUSrcB==2)?ext_imm:
    (ext_imm << 2);

```

6. ALU 모듈로 alu 연산을 해줍니다.

7. alu_zero 값을 설정해주고, 컨트롤 값에 따라서 메모리에 적을 주소와 값을 세팅해줍니다.

```
assign alu_zero = (alu_result == 32'b1);
assign wr_addr = SavePC ? 5'd31 : (RegDst ? rd : rt);
assign wr_data = SavePC ? ALUOut : (MemtoReg ? MDR : ALUOut);
```

III. CPU.v 동작 과정 설명

1. 명령어에 따라 컨트롤 값을 가져옵니다. 이때 처음 명령어는 어떠한 값이든 상관없이 IF 단계에서는 컨트롤 값이 고정되어 있으므로 상관없습니다.
2. IR을 split하고 signExtend 해주고 메모리에서 명령어를 가져올 것인지 데이터를 읽을 것인지 정합니다.
3. 메모리에서 값을 가져와서 오거나 전 clk에서 세팅된 값들로 메모리에 값을 write 합니다.
4. 레지스터에서 데이터를 가져오고 이를 가지고 operand를 설정합니다.
5. operand가 설정되면 alu 연산이 돌고 pc를 업데이트한다면 업데이트 해줍니다.
6. MEM 단계라면 3번 과정을 해주고 WB 단계라면 레지스터에 값을 써줍니다.
7. 이러한 과정들을 IF→ID→EX→MEM→WB 단계를 도는데 명령어에 따라 각각 다른 FSM을 돌며 각 단계별로 다른 컨트롤 값을 가져 그 컨트롤 값을 통해 리소스를 공유하며 multicycle로 올바르게 동작할 수 있도록 했습니다.

Verilog 동작 확인

.mem 파일을 읽어오는 경로를 ../cpp/ 로 변경하여 편하게 읽어오도록 하였습니다.

```
$readmemh("../cpp/reference_mem.mem", memory);
$readmemh("../cpp/reference_reg.mem", register_file);
```

I. iverilog로 컴파일한 파일 실행

1. testcase1~7을 dump한 hex파일을 cpp로 실행할 때마다 initial_reg.mem, initial_mem.mem, reference_reg.mem, reference_mem.mem 파일을 생성해줍니다.
2. 각 생성된 파일들을 verilog로 돌려 최종 reg와 mem에 저장된 값들이 동일하다면 성공적으로 success를 출력하는지 확인했습니다.
3. 결과 모두 성공적으로 동작하는 것을 확인했습니다.
4. 이때 \$write() 함수로 각 모듈과 부분들마다 log를 남기도록해서 올바르게 동작하는지 확인했습니다.

```
> iverilog -o cpu ALU.v CPU.v CPU_tb.v CTRL.v GLOBAL.v MEM.v RF.v

> ./cpu
VCD info: dumpfile cpu.vcd opened for output.
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 24895000 (1ps)

> ./cpu
VCD info: dumpfile cpu.vcd opened for output.
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 7505000 (1ps)

> ./cpu
VCD info: dumpfile cpu.vcd opened for output.
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 5805000 (1ps)

> ./cpu
VCD info: dumpfile cpu.vcd opened for output.
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 28205000 (1ps)

> ./cpu
VCD info: dumpfile cpu.vcd opened for output.
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 12635000 (1ps)

> ./cpu
VCD info: dumpfile cpu.vcd opened for output.
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 8225000 (1ps)

> ./cpu
VCD info: dumpfile cpu.vcd opened for output.
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 20335000 (1ps)
```

Trouble Shooting

1. 그냥 CPU에 각 명령어들의 STATE를 알게하여 IF문이나 Switch 문으로 각 단계별로 동작을 다르게 구현하는 것은 어떠한지 고민했습니다. 그런데 이렇게 구현하게 된다면 CPU가 클럭마다 다른 STATE를 돌며 동작을 할때 지나가는 회로가 다르게 구현이 되거나 아니면 이를 관리하기 위해서 더 많은 wire가 필요할 것이라고 생각했습니다. 또한 Multicycle의 의미에서 벗어나는 구현이라 생각하여 Control Unit만 STATE를 알고 이에 따라 컨트롤을 내뱉는 형태로 구현하였습니다.
2. JR과 JUMP는 ID에서 이미 nextPC 값이 완성이 되어 ID 단계에서 넘어가도록 구현하려 했습니다.이렇게 구현을 하다보니 Branch는 EX 단계에서 jump를 하고 나머지는 ID 단계에서 jump를 하여 코드로 구현할 때 일관성이 떨어지는 부분이 생긴다고 생각하여 모두 EX 단계에서 PCWrite을 하도록 했습니다.
3. 이번 과제에서 가장 핵심인 부분은 FSM으로 각 단계를 나누어 명령어마다 다른 사이클을 돌며 latency를 줄이는 것이지만 추가적인 과제로 resource를 share하도록 하는 것도 있었습니다. 이를 구현하기 위해서 PC+4, A+B 등 모든 연산을 ALU Unit을 사용하여 했습니다. 이렇게 구현하다 보니 ALU Unit이 비동기로 구현이 되어있어 input이 바뀔 때 마다 동작하도록 되어있었습니다. 그래서 PC값이 변경되거나 operand1이나 operand2가 clk이외에 RF 동작, MEM 동작 등으로 변경되어 ALU에 의미없는 값이 나오는 경우들이 있었습니다. 이를 최대한 줄여 타이밍을 일치시켰지만 $PC \leq PC + 4$ 를 하는 과정에서 PC가 덮여쓰이는 부분이 있어 ALU가 2번 동작하는 문제가 있었습니다. 이에 대해 고민하다 실제로 하드웨어 동작에서 결국엔 최종적으로 즉 마지막에 들어간 값이 계산된 결과를 사용하는 것이기 때문에 문제가 없을 것이라 생각했습니다. 만약 정확하게 한번만 동작해야한다면 ALU를 매 clk마다 동기로 동작하게 변경하여야 하는데 오히려 이 부분이 실제 HW 구현과 의미상 달라지는 부분이 있다고 생각하여 비동기로 유지하고 구현했습니다.

4. PC 값이 ALU 연산을 하자마자 변경되면서 ALU가 한 번 더 동작하는 것을 방지하기 위해서 nextPC를 사용했는데 오히려 이 때문에 PC에 값을 쓰이는 타이밍이 밀리면서 Control flow 명령어들이 올바르게 동작하지 못하는 문제가 있었습니다. 이를 PC를 업데이트하는 시점을 수정하여 해결하였습니다.